Reviewer Pack — Matroid Representation Rigidity and Disjointness Communicati…

Entry 4cd1defb419b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 03:40:57 UTC

Matroid Representation Rigidity and Disjointness Communication Complexity

Entry ID: `4cd1defb419b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 03:40:57 UTC

1. Conjecture

Field A (mathematical branch): Matroid Representations over Finite Fields **Field B** (complexity object): Communication Complexity of the Disjointness Problem

Statement:

> For any matroid M representable over a finite field F_q, the deterministic communication complexity of the disjointness problem for the matroid's characteristic vectors is Ω(log n), where n is the size of the ground set. This lower bound is achieved when the matroid's representation is rigid under field automorphisms.

Rationale (proposer's reasoning):

> Matroid representations over finite fields encode combinatorial structures with algebraic constraints. By analyzing the rigidity of these representations under field automorphisms, we can derive communication complexity lower bounds for the disjointness problem, which is a fundamental problem in communication complexity. The rigidity condition ensures that the matroid's structure cannot be simplified via field operations, linking algebraic properties to combinatorial complexity.

Taxonomy category: `ACC_SIPSER` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `70130c9496278d78`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random binary matroid with ground set size n
    n = random.randint(5, 40)
    elements = list(range(n))
    rank = random.randint(1, n)
    independent_sets = []
    for _ in range(2**rank):
        subset = random.sample(elements, rank)
        if all(len(set(subset) & set(iset)) <= 1 for iset in independent_sets):
            independent_sets.append(subset)

    # Compute the characteristic vectors
    char_vectors = [[0] * n for _ in range(2**rank)]
    for i, subset in enumerate(independent_sets):
        for j in subset:
            char_vectors[i][j] = 1

    # Simulate the disjointness problem using a protocol that checks for disjointness via bitwise
    communication_complexity = 0
    for _ in range(100):  # Test with 100 random pairs of vectors
        i, j = random.sample(range(2**rank), 2)
        if any(char_vectors[i][k] & char_vectors[j][k] for k in elements):
            communication_complexity += math.log2(n) + 1

    # Verify if the complexity meets Ω(log n)
    conjecture_holds = communication_complexity >= 0.5 * math.log2(n)

    return {
        "metric_name": "communication_complexity",
        "metric_value": communication_complexity,
        "instances_tested": 100,
        "conjecture_holds": conjecture_holds,
```

```
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]  # Default to first

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    mean_complexity = sum(r["metric_value"] for r in results) / len(results)
    std_complexity = math.sqrt(sum((r["metric_value"] - mean_complexity)**2 for r in results) / le
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_complexity} std={std_complexity} support_fraction={su
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_complexity} std={std_complexity} support_fraction={su
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing verification of support fraction or counterexamples. | next: Increase timeout duration and re-run the experiment with deterministic seed configurations

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	137528
2	propose	ollama_remote	qwen3:8b	0	0	155530
3	preregistration	ollama_remote	qwen3:8b	0	0	31680
4	novelty	ollama_remote	qwen3:8b	0	0	27548
5	novelty	ollama_remote	qwen3:8b	0	0	20596
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38757
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10910
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8766
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8197
10	judge	ollama_remote	qwen3:8b	0	0	18676

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 458187 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4cd1defb419b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4cd1defb419b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4cd1defb419b.tar.gz` (if generated)